MDP-DSL

A Domain-Specific Language for Markov Decision Processes

Complete Technical Reference & Tutorial

From zero prerequisites to full compiler & AI solver

Author:	Charvit Rajani
Roll Number:	240102028
Institution:	IIT Guwahati
Date:	April 10, 2026
Language:	Pure C++17 — No external libraries
Phases:	5 of 5 — All delivered
Total Code:	1,993 lines

Compiled with LaTeX · Pure C++17 · No external libraries · All 5 phases delivered

Abstract

This document is a complete, self-contained technical reference for the **MDP-DSL** project: a custom Domain-Specific Language (DSL) designed to define and solve **Markov Decision Processes** (MDPs) — the mathematical framework that underlies modern AI decision-making.

The project was developed in five sequential phases, each building on the last:

- **Phase 1 (Parser):** Reads a `.mdp` text file and constructs an Abstract Syntax Tree (AST) in memory.
- **Phase 2 (Semantic Validator):** Inspects the AST against seven mathematical and logical correctness checks, blocking the pipeline on errors.
- **Phase 3 (Value Iteration Solver):** Runs the Bellman equation iteratively to compute optimal value functions and policies.
- **Phase 4 (GridWorld Case Study):** Demonstrates the DSL on the classic 4×3 Russell & Norvig GridWorld and compares results against a hand-coded C++ implementation.
- **Phase 5 (Final Polish):** Edge-case hardening, README, and final documentation.

The report is structured so that **a reader with zero prior knowledge** of compilers, programming language theory, or artificial intelligence can read from front to back and emerge with a thorough understanding of all concepts. Every technical term is defined from first principles, illustrated with real-world analogies, and connected to the concrete C++ code that implements it.

Contents

Abstract **1**

1 Motivation and Background **5**

1.1 The Problem: Describing AI Decision Problems Safely 5

1.2 The Solution: A Domain-Specific Language 6

1.3 Why This Matters: Separation of Concerns 6

1.4 Project Scope and Deliverables 7

2 Mathematical Foundations: Markov Decision Processes **8**

2.1 Probability Theory: A Minimal Refresher 8

2.1.1 Events and Probability 8

2.1.2 Probability Distributions 8

2.1.3 Expected Value 9

2.2 The Markov Property 10

2.3 Markov Decision Processes: Formal Definition 10

2.3.1 Component-by-Component Explanation 10

2.4 The Goal: Optimal Policy 12

2.5 Value Functions 13

2.5.1 The Q-Function (Action-Value Function) 13

2.6 The Bellman Optimality Equation 13

2.7 Absorbing States 14

3 The `.mdp` Language: Grammar & Syntax **15**

3.1 Design Philosophy 15

3.2 The Five Keywords 15

3.2.1 Lexical Rules 15

3.3 A Minimal Example: `gridworld_simple.mdp` 16

3.3.1 Reading the Example 17

3.4 The Complete 4×3 GridWorld: `gridworld_4x3.mdp` 17

3.4.1 Layout 17

3.4.2 The Stochastic Slip Model 18

3.4.3 Transition Snippet 18

3.5 Edge Case Files . . . 19
3.5.1 `invalid_mdp.mdp` — Validator Tests . . . 19
3.5.2 `edge_gamma1.mdp` — Divergence Test . . . 19

4 Compiler Architecture: How the Code Works 20
4.1 The Compiler Pipeline . . . 20
4.2 Data Structures: The Abstract Syntax Tree . . . 20
4.2.1 The `Transition` Struct . . . 21
4.2.2 The `MDP_AST` Struct . . . 21
4.2.3 Data Structure Rationale . . . 21
4.3 Phase 1: The Parser Engine . . . 22
4.3.1 Algorithm Overview . . . 22
4.3.2 Tokenisation: How `std::istringstream` Works . . . 22
4.3.3 Error Handling Strategy . . . 23
4.3.4 The Parser Source Code . . . 23
4.4 Phase 2: The Semantic Validator . . . 24
4.4.1 Syntax vs. Semantics . . . 24
4.4.2 The ValidationResult Struct . . . 24
4.4.3 The Seven Validation Checks . . . 25
4.4.4 Single-Pass Architecture . . . 26
4.4.5 Validator Test Results . . . 26
4.5 Memory Safety and C++17 RAII . . . 27

5 The Value Iteration Solver 28
5.1 Overview . . . 28
5.2 The Bellman Operator . . . 28
5.3 Why Value Iteration Converges . . . 28
5.4 The Transition Index: An Acceleration Structure . . . 29
5.5 The Value Iteration Algorithm . . . 30
5.5.1 The Core Loop in C++ . . . 30
5.6 Complexity Analysis . . . 32
5.7 Test Results: gridworld_simple.mdp . . . 32
5.7.1 Solver Output . . . 32
5.7.2 Manual Mathematical Verification . . . 32

6 GridWorld Case Study: DSL vs. Hardcoded 34
6.1 The 4×3 GridWorld Problem . . . 34
6.1.1 Problem Specification . . . 34
6.2 DSL Solution: gridworld_4x3.mdp . . . 35
6.3 Hardcoded C++ Solution . . . 36

6.4 Quantitative Comparison . . . 36
6.5 Solver Results: The Optimal Policy . . . 36
6.5.1 Policy Visualisation . . . 37
6.5.2 Policy Analysis . . . 37

7 Edge Cases and Testing **39**
7.1 Test Suite Overview . . . 39
7.2 The $\gamma = 1.0$ Edge Case . . . 39
7.3 The Missing File Edge Case . . . 40
7.4 The Empty File Edge Case . . . 40
7.5 Duplicate State Declarations . . . 40
7.6 Malformed TRANSITION Lines . . . 40

8 Lessons Learned and Future Work **41**
8.1 Engineering Lessons . . . 41
8.1.1 Separation of Concerns Pays Off . . . 41
8.1.2 Validation Is Not Optional . . . 41
8.1.3 Floating-Point Equality Is Never Safe . . . 41
8.1.4 The Transition Index Was Critical . . . 41
8.1.5 RAII Prevents Entire Bug Classes . . . 42
8.2 Future Work . . . 42
8.2.1 State-Action Rewards: $R(s, a)$ . . . 42
8.2.2 Policy Iteration . . . 42
8.2.3 Graphical Output . . . 42
8.2.4 Larger GridWorlds and Performance Testing . . . 42
8.2.5 Interactive REPL Mode . . . 42
8.2.6 Multi-Armed Bandits . . . 43
8.3 Conclusion . . . 43

A Complete File Inventory **44**

B Quick-Reference: The .mdp Grammar **45**

C Compilation and Execution Guide **46**
C.1 Prerequisites . . . 46
C.2 Compilation . . . 46
C.3 Running the Compiler . . . 46
C.4 Expected Exit Codes . . . 47

D Glossary **48**

Chapter 1

Motivation and Background

1.1 The Problem: Describing AI Decision Problems Safely

When an engineer wants to teach a computer to make decisions — to navigate a maze, to manage a robot, to schedule resources — they typically reach for a general-purpose programming language such as C++, Python, or Java. The problem is that general-purpose languages are not designed for this specific task. They give the programmer enormous power, but that power comes with enormous responsibility: every index into an array, every probability value, every state name must be managed by hand, with no safety net.

Consider this scenario: you are modelling a robot navigating a warehouse. The robot can be in any of 200 locations (states), can attempt any of 8 directions (actions), and movement is uncertain — the robot might slip sideways. This gives $200 \times 8 = 1600$ distinct (state, action) pairs, each with 3 or more probabilistic outcomes: at least 4800 individual probability values.

If a single one of those 4800 values is slightly wrong — say, the probabilities for (location 47, move North) sum to 0.97 instead of 1.0 — the solver will produce a subtly incorrect policy. The robot will appear to work but will make systematically suboptimal decisions. The bug is silent: no error message, no crash, just a wrong answer.

Important Note

This is the fundamental problem that MDP-DSL solves. By moving the *definition* of the MDP into a structured text file with automatic validation, errors are caught **before** the solver ever runs, not after weeks of debugging mysterious AI behaviour.

1.2 The Solution: A Domain-Specific Language

Definition: Domain-Specific Language (DSL)

A **Domain-Specific Language** (DSL) is a small, focused programming language designed to solve problems in one particular domain. Unlike general-purpose languages (C++, Python) which can express any computation, a DSL restricts what can be said, and in doing so gains the power to *automatically verify* that what is said is correct.

Examples of DSLs you already know:

- `SQL` — designed for database queries, automatically enforces table schemas
- `HTML` — designed for web page structure, automatically detects unclosed tags
- `Makefile` — designed for build systems, automatically tracks dependencies
- **MDP-DSL** — designed for decision problems, automatically validates probability sums

MDP-DSL provides a plain-text `.mdp` file format with five keywords. The user writes a description of their decision problem; a compiler reads it, checks it, solves it, and prints the optimal strategy. The user never writes a loop, never manages an array index, and never worries about floating-point probability sums — the compiler handles all of that automatically.

1.3 Why This Matters: Separation of Concerns

The most important engineering principle at work in this project is **separation of concerns**: keeping distinct responsibilities in distinct places.

In a traditional hardcoded approach:

- **What** the problem is (states, transitions, rewards) — tangled into the code
- **How** to solve it (the Value Iteration algorithm) — tangled into the code
- **Whether** it is correct (validation) — absent entirely

In the DSL approach:

- **What** the problem is — lives in a `.mdp` file (pure data, no algorithm)
- **How** to solve it — lives in the compiler (reused for every new problem)
- **Whether** it is correct — automatically checked by Phase 2

To define a completely new AI decision problem with the DSL approach, the user writes

one new `.mdp` file. The compiler is untouched. With the hardcoded approach, the user must rewrite the entire program from scratch.

1.4 Project Scope and Deliverables

Table 1.1: Project phases, dates, and deliverables

Phase	Name	Date	Key Deliverable
1	Parser Engine	March 29, 2026	`parser.cpp` — reads `.mdp` → AST
2	Semantic Validator	April 4, 2026	`mdp_compiler.cpp` — 7 correctness checks
3	Value Iteration Solver	April 10, 2026	Full pipeline + solver
4	GridWorld Case Study	April 10, 2026	DSL vs. hardcoded comparison
5	Final Polish	April 10, 2026	This document + edge case tests

Chapter 2

Mathematical Foundations: Markov Decision Processes

This chapter defines every mathematical concept used in the project from first principles. A reader with no background in probability theory or artificial intelligence should read this chapter carefully before proceeding.

2.1 Probability Theory: A Minimal Refresher

2.1.1 Events and Probability

Definition: Probability

A **probability** is a number in $[0, 1]$ that measures how likely an event is.

- $P = 0$ means the event is *impossible*.
- $P = 1$ means the event is *certain*.
- $P = 0.7$ means the event happens in roughly 70 out of 100 trials.

2.1.2 Probability Distributions

When we have a set of mutually exclusive outcomes that cover all possibilities, their probabilities must sum to exactly 1.0. This is called a **probability distribution**.

Example: Rolling a Fair Die

A fair six-sided die has six outcomes: $\{1, 2, 3, 4, 5, 6\}$. Each has probability $1/6 \approx 0.167$.

$$P(1) + P(2) + P(3) + P(4) + P(5) + P(6) = 6 \times \tfrac{1}{6} = 1.0$$

This is a valid probability distribution. If one face had $P = 0.2$ and another $P = 0.1$, they would sum to $\neq 1$ — an invalid distribution.

IEEE 754 Floating-Point Imprecision

Computers store numbers in binary (base 2). Simple decimals like 0.1 cannot be represented exactly in binary, just as $\frac{1}{3}$ cannot be written exactly in base 10:

$$0.1 + 0.2 = \underbrace{0.30000000000000004}_{\text{actual computer result}} \neq 0.3$$

This means we can **never check** `sum == 1.0`. Instead, we check:

$$|sum - 1.0| < \varepsilon \quad \text{where } \varepsilon = 10^{-9}$$

For typical MDPs with 2–10 transitions per (state, action) pair, accumulated rounding error stays comfortably below 10^{-9}.

2.1.3 Expected Value

Definition: Expected Value

Given a random variable X with outcomes $\{x_1, x_2, \ldots, x_n\}$ and probabilities $\{p_1, p_2, \ldots, p_n\}$, the **expected value** (or *expectation*) is:

$$\mathbb{E}[X] = \sum_{i=1}^{n} p_i \cdot x_i$$

It is the probability-weighted average outcome — the value you would expect "on average" over many trials.

Example: Expected Winnings

A game: flip a coin. Heads wins \$3, tails loses \$1. Expected winnings:

$$\mathbb{E}[\text{winnings}] = 0.5 \times 3 + 0.5 \times (-1) = 1.5 - 0.5 = \$1.00$$

On average, you gain \$1 per game — even though each individual game ends in \$3 or -\$1.

2.2 The Markov Property

Definition: The Markov Property

A stochastic process has the **Markov property** if the future depends *only* on the present state, not on the history of how that state was reached:

$$P(\text{next state} \mid \text{current state, all past states}) = P(\text{next state} \mid \text{current state})$$

Informally: *the past is irrelevant; only where you are now matters.*

Real-world analogy: Chess is approximately Markovian. The value of a board position depends on what pieces are where now — not on the specific sequence of moves that created that position. If two different sequences of moves lead to the same board, the optimal strategy is identical from both.

Counterexample (non-Markovian): Blackjack *without* reshuffling is not Markovian, because which cards remain in the deck depends on every card played so far. A card counter's strategy depends on history, not just the current hand.

2.3 Markov Decision Processes: Formal Definition

Definition: Markov Decision Process (MDP)

A **Markov Decision Process** (MDP) is a tuple $\langle S, A, T, R, \gamma \rangle$ where:

- S — a finite set of **states** (all possible situations the agent can be in)
- A — a finite set of **actions** (all possible choices the agent can make)
- $T : S \times A \times S \to [0, 1]$ — the **transition function**, where $T(s, a, s')$ is the probability of arriving at state s' after taking action a in state s
- $R : S \to \mathbb{R}$ — the **reward function**, where $R(s)$ is the immediate numerical reward for being in state s
- $\gamma \in [0, 1]$ — the **discount factor**, controlling how much future rewards are valued

Stochasticity constraint: For every state $s \in S$ and action $a \in A$:

$$\sum_{s' \in S} T(s, a, s') = 1.0$$

2.3.1 Component-by-Component Explanation

States (S)

A state is a complete description of the agent's current situation. In GridWorld, each cell (r, c) is a state. In a robot controller, the state might be the robot's (x, y, θ) position and orientation. In a chess-playing AI, the state is the complete board configuration.

The set S must be finite and exhaustive — every possible situation must correspond to exactly one state. Our DSL enforces this: you declare every state with a `STATE:` line.

Actions (A)

An action is a choice the agent makes. In GridWorld, the actions are the four compass directions: {Up, Down, Left, Right}. The set A must also be finite. Declaring actions explicitly (via `ACTION:` lines) allows the validator to catch typos in transition rules.

Transition Function (T)

The transition function is the heart of the MDP. It encodes *uncertainty*: the world is stochastic, so the same action from the same state doesn't always lead to the same outcome.

Example: Stochastic Robot Movement

A robot tries to move North from location $(3, 2)$. Due to wheel slip:

- With probability 0.80: arrives at $(3, 3)$ (correct — went North)
- With probability 0.10: arrives at $(2, 2)$ (slipped West)
- With probability 0.10: arrives at $(4, 2)$ (slipped East)

In MDP notation: $T((3, 2), \text{North}, (3, 3)) = 0.80$, $T((3, 2), \text{North}, (2, 2)) = 0.10$, $T((3, 2), \text{North}, (4, 2)) = 0.10$.

Reward Function (R)

The reward function is how we tell the agent *what we want it to achieve*. Positive rewards attract the agent; negative rewards (penalties) repel it. In GridWorld:

- $R(\text{Goal}) = +1.0$ — reaching the goal is highly desirable
- $R(\text{Trap}) = -1.0$ — entering the trap is highly undesirable
- $R(s) = -0.04$ for all other states — a small living penalty that encourages the agent to reach the goal quickly rather than wandering

Discount Factor (γ)

The discount factor $\gamma \in [0, 1]$ controls the agent's *time preference* — how much it values future rewards compared to immediate ones.

Definition: Discounted Return

The **discounted return** starting from state s_0 along a trajectory $s_0, s_1, s_2, \ldots$ is:

$$G = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots = \sum_{t=0}^{\infty} \gamma^t R(s_t)$$

- $\gamma = 0$: agent is completely myopic — only cares about immediate reward
- $\gamma = 0.9$: a reward one step away is worth 90%, two steps away 81%, and so on
- $\gamma = 1$: all future rewards count equally (can cause divergence in infinite-horizon MDPs)

The discount factor also ensures the infinite sum G converges, as long as $\gamma < 1$:

$$|G| \leq \sum_{t=0}^{\infty} \gamma^t |R_{\max}| = \frac{|R_{\max}|}{1 - \gamma} < \infty$$

2.4 The Goal: Optimal Policy

Definition: Policy

A **policy** $\pi : S \to A$ is a mapping from states to actions. It specifies what action the agent should take in every possible situation.
An **optimal policy** π^* maximises the expected discounted return from every starting state simultaneously.

Finding the optimal policy is the core computational problem that MDP-DSL solves. The algorithm used is **Value Iteration**, explained in depth in Chapter 5.

2.5 Value Functions

Definition: Value Function

The **value function** $V^\pi : S \to \mathbb{R}$ under policy π gives the expected discounted return of following π starting from state s:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t) \;\middle|\; s_0 = s \right]$$

The **optimal value function** $V^* : S \to \mathbb{R}$ gives the maximum achievable expected return from each state:

$$V^*(s) = \max_\pi V^\pi(s)$$

2.5.1 The Q-Function (Action-Value Function)

Definition: Q-Function

The **Q-function** (or *action-value function*) $Q(s, a)$ gives the expected discounted return of taking action a in state s and then following the optimal policy:

$$Q(s, a) = \sum_{s' \in S} T(s, a, s') \cdot V^*(s')$$

The optimal policy chooses the action with the highest Q-value:

$$\pi^*(s) = \arg\max_{a \in A} Q(s, a)$$

2.6 The Bellman Optimality Equation

Theorem: Bellman Optimality Equation

The optimal value function V^* satisfies the **Bellman Optimality Equation**:

$$V^*(s) = R(s) + \gamma \max_{a \in A} \sum_{s' \in S} T(s, a, s') \cdot V^*(s')$$

This is a *recursive* definition: the value of state s equals its immediate reward plus the discounted value of the best next state reachable from s.

Intuition: Imagine you are at a junction and must choose a road. The value of your current position equals your immediate bonus for standing here, plus γ times the value of

the best destination road leads to (averaged over the stochastic outcomes of each possible choice).

The Bellman equation is a *system of* $|S|$ *simultaneous equations* — one per state. Solving it exactly is computationally expensive, but iterating towards the solution (Value Iteration) is highly efficient.

2.7 Absorbing States

Definition: Absorbing State

A state s is **absorbing** if $T(s, a, s) = 1.0$ for all a — once entered, the agent can never leave. Terminal states like the goal and the trap in GridWorld are modelled as absorbing.

For an absorbing state with reward $R(s)$, the agent receives $R(s)$ forever (discounted):

$$V^*(s) = R(s) + \gamma R(s) + \gamma^2 R(s) + \cdots = \frac{R(s)}{1-\gamma}$$

Example: GridWorld Terminal Values

With $\gamma = 0.9$:

$$V^*(\text{Goal}) = \frac{+1.0}{1-0.9} = \frac{1.0}{0.1} = 10.0$$

$$V^*(\text{Trap}) = \frac{-1.0}{1-0.9} = \frac{-1.0}{0.1} = -10.0$$

These large magnitudes reflect the infinite horizon: the agent is stuck in these states permanently, accumulating reward (or penalty) forever.

Chapter 3

The `.mdp` Language: Grammar & Syntax

3.1 Design Philosophy

The `.mdp` language is designed around four principles:

1. **Readability:** A human should be able to read a `.mdp` file and immediately understand the MDP being described.
2. **Simplicity:** Exactly five keywords. One statement per line. No nesting, no brackets, no semicolons.
3. **Safety:** Every piece of information is explicit, allowing the compiler to verify correctness.
4. **Portability:** Plain ASCII text. Opens in any text editor on any platform.

3.2 The Five Keywords

Table 3.1: Complete keyword reference for the `.mdp` language

Keyword	Syntax	Purpose
`STATE:`	`STATE: <name>`	Declare a state
`ACTION:`	`ACTION: <name>`	Declare an action
`TRANSITION:`	`TRANSITION: <src> <act> <dst> <prob>`	Define one outcome
`REWARD:`	`REWARD: <state> <value>`	Assign state reward
`DISCOUNT:`	`DISCOUNT: <gamma>`	Set discount factor

3.2.1 Lexical Rules

- Lines beginning with `#` are **comments** and are completely ignored.

- Blank lines are silently skipped.
- Keywords are **case-insensitive**: `state:`, `State:`, and `STATE:` all work.
- State and action **names are case-sensitive**: `Start` $\neq$ `start`.
- Leading and trailing whitespace on every line is trimmed automatically.
- Tokens within a line are separated by one or more whitespace characters.

3.3 A Minimal Example: `gridworld_simple.mdp`

Listing 3.1: A simple 4-state MDP demonstrating all five keywords

```
# A simple 4-state MDP for testing
# States: Start, Middle, Goal, Trap

STATE: Start
STATE: Middle
STATE: Goal
STATE: Trap

ACTION: MoveRight
ACTION: MoveLeft
ACTION: Stay

TRANSITION: Start  MoveRight  Middle  0.8
TRANSITION: Start  MoveRight  Start   0.2
TRANSITION: Middle MoveRight  Goal    0.7
TRANSITION: Middle MoveRight  Trap    0.3
TRANSITION: Middle MoveLeft   Start   0.9
TRANSITION: Middle MoveLeft   Middle  0.1
TRANSITION: Goal   Stay       Goal    1.0
TRANSITION: Trap   Stay       Trap    1.0

REWARD: Start   0.0
REWARD: Middle  1.0
REWARD: Goal    100.0
REWARD: Trap   -50.0

DISCOUNT: 0.9
```

3.3.1 Reading the Example

Let us decode each section:

States Four states: `Start`, `Middle`, `Goal`, `Trap`. These are all possible locations for the agent.

Actions Three actions. Note: not all actions are available from every state. If no transition is defined for a (state, action) pair, that action is simply unavailable from that state.

Transitions Reading line by line:

- From `Start`, `MoveRight`: 80% chance of reaching `Middle`, 20% chance of staying at `Start`. (Probabilities: $0.8 + 0.2 = 1.0$ ✓)
- From `Middle`, `MoveRight`: 70% chance of `Goal`, 30% chance of `Trap`. (Probabilities: $0.7 + 0.3 = 1.0$ ✓)
- `Goal` and `Trap` are absorbing: they loop to themselves with probability 1.0.

Rewards `Goal` gives $+100$ (large positive reward), `Trap` gives -50 (large negative penalty).

Discount $\gamma = 0.9$ — standard in most RL textbooks.

3.4 The Complete 4×3 GridWorld: `gridworld_4x3.mdp`

3.4.1 Layout

The 4×3 GridWorld is the canonical benchmark from Russell & Norvig's *Artificial Intelligence: A Modern Approach*. Its layout is:

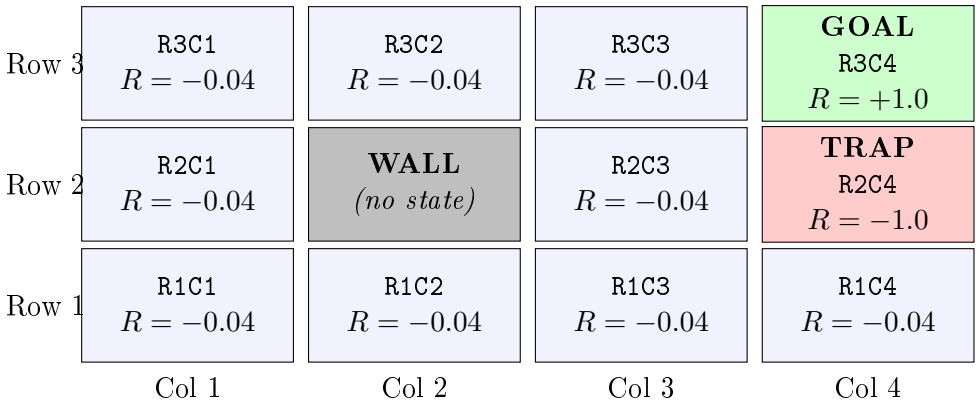

Figure 3.1: The 4×3 GridWorld layout. Naming: `RrCc` = Row r, Column c.

3.4.2 The Stochastic Slip Model

Movement in this GridWorld is uncertain. When the agent intends to go in a direction:

- **80%:** moves in the intended direction
- **10%:** slips left (perpendicular to intended direction)
- **10%:** slips right (perpendicular to intended direction)

If the movement would take the agent into a wall or off the grid, the agent bounces back to its current cell.

Table 3.2: Perpendicular directions for the slip model

Intended	10% Slip Left	10% Slip Right
Up	Left	Right
Down	Right	Left
Left	Down	Up
Right	Up	Down

3.4.3 Transition Snippet

Listing 3.2: Transitions for state R1C1 (bottom-left corner)

```
# R1C1 (bottom-left corner)
# Up: intended=R2C1(80%), slip-left=boundary->R1C1(10%), slip-right=R1C2(10%)
TRANSITION: R1C1 Up   R2C1  0.8
TRANSITION: R1C1 Up   R1C1  0.1
TRANSITION: R1C1 Up   R1C2  0.1

# Down: intended=boundary->stay(90%), slip-right=R1C2(10%)
TRANSITION: R1C1 Down R1C1  0.9
```

```
TRANSITION: R1C1 Down R1C2  0.1

# Left: intended=boundary->stay(90%), slip-right=R2C1(10%)
TRANSITION: R1C1 Left R1C1  0.9
TRANSITION: R1C1 Left R2C1  0.1

# Right: intended=R1C2(80%), slip-left=R2C1(10%), slip-right=boundary->stay(10%)
TRANSITION: R1C1 Right R1C2  0.8
TRANSITION: R1C1 Right R2C1  0.1
TRANSITION: R1C1 Right R1C1  0.1
```

The full `gridworld_4x3.mdp` file contains 11 states, 4 actions, and 105 transition rules.

3.5 Edge Case Files

3.5.1 `invalid_mdp.mdp` — Validator Tests

This file deliberately contains errors to test the Phase 2 validator:

- A transition referencing `Trap` before it is declared as a state
- Transitions using `MoveLeft` which is never declared as an action
- A (state, action) pair whose probabilities sum to 0.6, not 1.0
- A `REWARD` line referencing the non-existent state `Phantom`

3.5.2 `edge_gamma1.mdp` — Divergence Test

This file uses $\gamma = 1.0$, the boundary case where future rewards have the same weight as immediate rewards. The geometric series $\sum_{t=0}^{\infty} 1^t \cdot R$ does not converge, so Value Iteration takes the full 10,000 iterations without converging — a deliberate stress test of the solver's safety cap.

Chapter 4

Compiler Architecture: How the Code Works

4.1 The Compiler Pipeline

The MDP-DSL compiler processes a `.mdp` file through a three-phase pipeline:

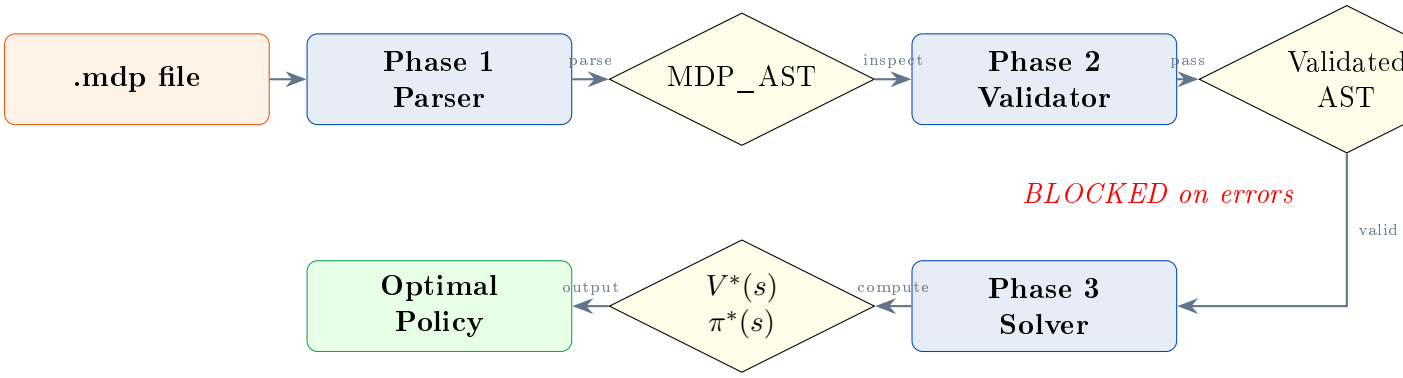

Figure 4.1: The three-phase MDP-DSL compiler pipeline

4.2 Data Structures: The Abstract Syntax Tree

Definition: Abstract Syntax Tree (AST)

An **Abstract Syntax Tree** is the structured, in-memory representation of parsed source code. After parsing, the raw text is gone — only the meaningful data remains, organised into labelled containers.

Analogy: A recipe written on a napkin (the `.mdp` file) gets reorganised into labelled jars: one for ingredients (states), one for techniques (actions), one for instructions (transitions), one for amounts (rewards).

4.2.1 The `Transition` Struct

Listing 4.1: The Transition struct — one probabilistic outcome

```
struct Transition {
    std::string source_state;  // State we depart from (e.g., "Start")
    std::string action;        // Action performed  (e.g., "MoveRight")
    std::string dest_state;    // State we arrive at (e.g., "Middle")
    double      probability;   // Probability in [0.0, 1.0]
};
```

Each `TRANSITION:` line in the `.mdp` file creates exactly one `Transition` object. The mathematical constraint $\sum_{s'} T(s, a, s') = 1.0$ is not enforced here — it is checked by the validator in Phase 2.

4.2.2 The `MDP_AST` Struct

Listing 4.2: The MDP_AST master container

```
struct MDP_AST {
    std::unordered_set<std::string>         states;
    std::unordered_set<std::string>         actions;
    std::vector<Transition>                 transitions;
    std::unordered_map<std::string,double>  rewards;
    double                                  discount_factor;

    MDP_AST() : discount_factor(0.9) {}  // default gamma
};
```

4.2.3 Data Structure Rationale

Table 4.1: Why each container type was chosen

Field	Type	Lookup	Rationale
`states`	`unordered_set`	$O(1)$	"Is state X declared?" — $O(1)$ membership test + auto-deduplication
`actions`	`unordered_set`	$O(1)$	Same as states
`transitions`	`vector`	$O(n)$	Multiple entries per (src,act) pair; insertion-order preserved
`rewards`	`unordered_map`	$O(1)$	"What is the reward for state X?" — $O(1)$ map lookup

4.3 Phase 1: The Parser Engine

4.3.1 Algorithm Overview

Algorithm 1 parseMDPFile(filepath)

```
Open filepath with std::ifstream
if file cannot be opened then
    Print fatal error, return empty AST
end if
Initialise empty MDP_AST ast
while getline reads a line do
    Increment line number counter
    Trim leading/trailing whitespace from line
    if line is empty or starts with # then
        continue (skip)
    end if
    Extract first token as keyword
    Convert keyword to uppercase
    if keyword == "STATE:" then Extract name, insert into ast.states
    else if keyword == "ACTION:" then Extract name, insert into ast.actions
    else if keyword == "TRANSITION:" then Extract src, act, dst, prob; append to ast.transitions
    else if keyword == "REWARD:" then Extract state name and value; store in ast.rewards
    else if keyword == "DISCOUNT:" then Extract gamma; clamp to [0, 1]; store in ast.discount_factor
    else Print warning about unknown keyword
    end if
end while
return ast
```

4.3.2 Tokenisation: How `std::istringstream` Works

The C++ standard library provides `std::istringstream`, which treats a string as a stream and extracts whitespace-separated tokens using the » operator.

Step-by-step tokenisation of a TRANSITION line

Input string: `"TRANSITION: Start MoveRight Middle 0.8"`

1. `iss » keyword` extracts `"TRANSITION:"`
2. `iss » src` extracts `"Start"`
3. `iss » act` extracts `"MoveRight"`
4. `iss » dst` extracts `"Middle"`
5. `iss » prob` extracts `0.8` (auto-converted from string to `double`)

If step 5 fails (e.g., missing probability), `iss.fail()` returns `true` and a warning is printed.

4.3.3 Error Handling Strategy

The parser uses a *report-and-continue* philosophy: on encountering a malformed line, it prints a warning to `std::cerr` and moves to the next line. It *never aborts* on the first error. This means one run reveals *all* syntax problems, not just the first one.

Table 4.2: Parser error types and responses

Error	Severity	Response
Duplicate state declaration	Warning	Duplicate ignored, parsing continues
Malformed TRANSITION (missing tokens)	Warning	Line skipped
Missing reward value	Warning	Line skipped
Probability outside $[0, 1]$	Warning	Value stored as-is (validator catches it)
Out-of-range discount factor	Warning	Value clamped to $[0, 1]$
Unknown keyword	Warning	Line skipped
File not found	Fatal Error	Program exits with code 1

4.3.4 The Parser Source Code

Listing 4.3: Core parser dispatch loop (from mdp_compiler.cpp)

```
while (std::getline(file, line)) {
    line_number++;
    std::string trimmed = trim(line);
    if (trimmed.empty() || trimmed[0] == '#') continue;

    std::istringstream iss(trimmed);
    std::string keyword;
    iss >> keyword;
    std::string keyword_upper = toUpper(keyword);

    if (keyword_upper == "STATE:") {
        std::string name; iss >> name;
        if (name.empty()) {
            std::cerr << "[WARNING] Line " << line_number
                      << ": STATE with no name.\n";
            continue;
        }
        auto [it, ok] = ast.states.insert(name);
```

```
        if (!ok)
            std::cerr << "[WARNING] Line " << line_number
                      << ": Duplicate state '" << name << "'.\n";
    }
    else if (keyword_upper == "TRANSITION:") {
        std::string src, act, dst; double prob = 0.0;
        iss >> src >> act >> dst >> prob;
        if (iss.fail() || src.empty()) {
            std::cerr << "[WARNING] Line " << line_number
                      << ": Malformed TRANSITION.\n";
            continue;
        }
        ast.transitions.emplace_back(Transition{src, act, dst, prob});
    }
    // ... (REWARD, DISCOUNT, ACTION handled similarly)
}
```

4.4 Phase 2: The Semantic Validator

4.4.1 Syntax vs. Semantics

Definition: Syntax vs. Semantics

Syntax = the structure of a statement. Are the right keywords present? Are enough tokens on each line?

Semantics = the *meaning* of a statement. Do the referenced names actually exist? Do the numbers satisfy mathematical constraints?

Example: The line `TRANSITION: Strat MoveRight Middle 0.8` passes the syntax check (4 tokens) but fails the semantic check (`Strat` was never declared). The Phase 2 validator catches this.

4.4.2 The ValidationResult Struct

Listing 4.4: ValidationResult — collects all errors and warnings

```
struct ValidationResult {
    std::vector<std::string> errors;    // Critical: must fix before solving
    std::vector<std::string> warnings;  // Suspicious: informational only

    bool isValid() const { return errors.empty(); }
};
```

Exceptions are not used deliberately. Collecting all messages in vectors lets the user see every problem in one run, rather than fixing them one at a time.

4.4.3 The Seven Validation Checks

The Seven Invariants of a Valid MDP

For an MDP defined in a `.mdp` file to be mathematically valid, the following must all hold:

1. Every `source_state` in a TRANSITION is declared with STATE:
2. Every `dest_state` in a TRANSITION is declared with STATE:
3. Every `action` in a TRANSITION is declared with ACTION:
4. For every (state, action) pair: $\sum_{s'} T(s, a, s') = 1.0$ within tolerance 10^{-9}
5. Every declared state has a REWARD (warning only — solver assumes 0.0)
6. Every declared state appears in at least one TRANSITION (warning only)
7. Every state in a REWARD is declared with STATE:

Checks 1–4 and 7 are **ERRORS** (block the solver). Checks 5–6 are **WARNINGS** (allow proceeding).

Check 4: Probability Sum Validation (Detail)

This is the most technically interesting check. The algorithm uses a *composite key* strategy:

Listing 4.5: Probability accumulator with composite key

```
// Key: "source_state|action" -> accumulated probability sum
std::unordered_map<std::string, double> prob_sums;

for (const auto& t : ast.transitions) {
    // Checks 1, 2, 3: reference integrity (omitted for brevity)

    // Accumulate probability for this (state, action) pair
    std::string pair_key = t.source_state + "|" + t.action;
    prob_sums[pair_key] += t.probability;
}

// After all transitions: check each sum
const double EPSILON = 1e-9;
for (const auto& [key, sum] : prob_sums) {
    if (std::abs(sum - 1.0) > EPSILON) {
        result.errors.push_back(
            "Probability sum for (" + key + ") = " +
```

```
            std::to_string(sum) + " (expected 1.0)");
    }
}
```

Why the pipe separator? Without it, keys `"AB"` + `"CD"` and `"A"` + `"BCD"` both produce `"ABCD"` — a collision. With the pipe: `"AB|CD"` and `"A|BCD"` are distinct.

4.4.4 Single-Pass Architecture

All transition-based checks (1, 2, 3, 4, and orphan tracking) are performed in *a single loop* over the transitions vector:

$$\text{Time complexity: } O(|T|) \text{ for the main loop, then } O(|S| \cdot |A|) \text{ for sum checks}$$

$$\text{Total: } O(|S| + |A| + |T|) \text{ — linear in the size of the MDP}$$

4.4.5 Validator Test Results

Listing 4.6: Output for invalid_mdp.mdp

```
ERRORS (5) -- MDP is INVALID:
  [ERROR 1] Undeclared destination state 'Trap'
  [ERROR 2] Undeclared action 'MoveLeft' in TRANSITION Middle MoveLeft Start
  [ERROR 3] Undeclared action 'MoveLeft' in TRANSITION Middle MoveLeft Middle
  [ERROR 4] Probability sum for (Start, Stay) = 0.600000
            (expected 1.0, deviation = 0.400000)
  [ERROR 5] REWARD references undeclared state 'Phantom'

WARNINGS (1):
  [WARN 1] State 'Goal' has no REWARD. Solver assumes 0.0.

VERDICT: MDP is INVALID
BLOCKED: Cannot proceed to Phase 3 (Solver).
```

All five deliberately planted errors were detected correctly.

4.5 Memory Safety and C++17 RAII

Key Insight: No Raw Pointers

The entire MDP-DSL codebase contains **zero raw pointers**, **zero** `new` calls, and **zero** `delete` calls.

Every container (`std::string`, `std::vector`, `std::unordered_map`, `std::unordered_set`) manages its own heap memory through **RAII** (Resource Acquisition Is Initialisation). When any of these objects goes out of scope, its destructor automatically frees all allocated memory.

This means there is *zero possibility of memory leaks* in this codebase by construction.

The `parseMDPFile` function returns the `MDP_AST` by value. The C++ compiler's **NRVO** (Named Return Value Optimisation) constructs the returned object directly in the caller's stack frame, eliminating any copy overhead.

Chapter 5

The Value Iteration Solver

5.1 Overview

Value Iteration is the classic dynamic programming algorithm for solving MDPs. It repeatedly applies the Bellman operator to every state simultaneously until the value function converges to V^*.

5.2 The Bellman Operator

Definition: Bellman Operator $\mathcal{B}$

The **Bellman operator** $\mathcal{B}$ maps a value function V to an updated value function $\mathcal{B}V$:

$$(\mathcal{B}V)(s) = R(s) + \gamma \max_{a \in A} \sum_{s' \in S} T(s, a, s') \cdot V(s')$$

Value Iteration repeatedly applies $\mathcal{B}$: starting from V_0 (e.g., all zeros), computing $V_1 = \mathcal{B}V_0$, $V_2 = \mathcal{B}V_1$, and so on.

5.3 Why Value Iteration Converges

Theorem: Contraction Mapping (Convergence Guarantee)

The Bellman operator $\mathcal{B}$ is a **contraction mapping** in the sup-norm when $\gamma < 1$:

$$\|\mathcal{B}V - V^*\|_\infty \leq \gamma \cdot \|V - V^*\|_\infty$$

Since $\gamma < 1$, each application of $\mathcal{B}$ reduces the distance to V^* by a factor of γ. By the **Banach Fixed-Point Theorem**, the sequence $V_0, V_1, V_2, \ldots$ converges to the unique fixed point V^*.

Convergence rate: After k iterations,

$$\|V_k - V^*\|_\infty \leq \gamma^k \cdot \|V_0 - V^*\|_\infty$$

For $\gamma = 0.9$: after 200 iterations, error $\leq 0.9^{200} \approx 7 \times 10^{-10}$ — below our threshold $\theta = 10^{-9}$.

5.4 The Transition Index: An Acceleration Structure

Without preprocessing, computing $Q(s, a) = \sum_{s'} T(s, a, s') \cdot V(s')$ requires scanning all $|T|$ transitions for each (s, a) query. Over a full iteration:

$$O(|S| \cdot |A| \cdot |T|) \text{ per iteration — extremely slow for large MDPs}$$

The solution is to build a **transition index** once before the main loop:

Listing 5.1: Building the transition index in O(|T|)

```
using DestProb = std::pair<std::string, double>;
std::unordered_map<std::string, std::vector<DestProb>> transition_index;

for (const auto& t : ast.transitions) {
    // Key: "source_state|action" -> list of (dest, prob) pairs
    std::string key = t.source_state + "|" + t.action;
    transition_index[key].emplace_back(t.dest_state, t.probability);
}
```

After indexing, computing $Q(s, a)$ is $O(k)$ where k is the (small, constant) number of outcomes for that (s, a) pair. A full iteration costs $O(|T|)$:

$$O(|T|) \text{ per iteration — linear in the size of the MDP}$$

Table 5.1: Transition index contents for gridworld_simple.mdp

Key	**Value (destinations)**
`"Start|MoveRight"`	`[("Middle", 0.8), ("Start", 0.2)]`
`"Middle|MoveRight"`	`[("Goal", 0.7), ("Trap", 0.3)]`
`"Middle|MoveLeft"`	`[("Start", 0.9), ("Middle", 0.1)]`
`"Goal|Stay"`	`[("Goal", 1.0)]`
`"Trap|Stay"`	`[("Trap", 1.0)]`

5.5 The Value Iteration Algorithm

Algorithm 2 Value Iteration (complete)

Require: Validated MDP_AST, threshold $\theta = 10^{-9}$, max_iter = 10000
Ensure: Optimal value function V^*, optimal policy π^*

```
Build transition index from ast.transitions                    ▷ O(|T|)
Build actions_at_state map                  ▷ which actions are defined per state
Initialise V[s] ← 0.0 for all s ∈ S
Initialise π[s] ← "" for all s ∈ S
for iter = 1 to max_iter do
    δ ← 0.0
    for each state s ∈ S do
        v_old ← V[s]
        R_s ← ast.rewards[s] (or 0.0 if undefined)
        best_Q ← −∞
        for each action a available at state s do
            Q ← 0.0
            for each ⟨s′, p⟩ in transition_index[s|a] do
                Q ← Q + p · V[s′]
            end for
            if Q > best_Q then
                best_Q ← Q; π[s] ← a
            end if
        end for
        V[s] ← R_s + γ · best_Q
        δ ← max(δ, |V[s] − v_old|)
    end for
    if δ < θ then
        break                                                  ▷ Converged
    end if
end for
return (V, π, iterations)
```

5.5.1 The Core Loop in C++

Listing 5.2: The Bellman update loop (from mdp_compiler_phase3.cpp)

```
for (int iter = 1; iter <= max_iter; ++iter) {
    double delta = 0.0;

    for (const auto& state : ast.states) {
        double old_value = V[state];
        double reward = (ast.rewards.count(state) > 0)
                        ? ast.rewards.at(state) : 0.0;

        double best_q = std::numeric_limits<double>::lowest();
        std::string best_action = "";

        // Iterate over all actions available at this state
        for (const auto& action : actions_at_state[state]) {
            double q = 0.0;
            std::string key = state + "|" + action;
            // Sum over all (dest, prob) pairs for this (state, action)
            for (const auto& [dest, prob] : transition_index[key])
                q += prob * V[dest];  // Expected future value

            if (q > best_q) {
                best_q = q;
                best_action = action;
            }
        }

        // Bellman equation: V(s) = R(s) + gamma * max_a Q(s,a)
        V[state] = reward + ast.discount_factor * best_q;
        policy[state] = best_action;

        // Track maximum change for convergence check
        delta = std::max(delta, std::abs(V[state] - old_value));
    }

    if (delta < theta) {
        // Converged!
        break;
    }
}
```

5.6 Complexity Analysis

Table 5.2: Time and space complexity of the Value Iteration solver

Component	**Complexity**	**Explanation**
Build transition index	$O(\|T\|)$	One pass over all transitions
Per iteration (Bellman updates)	$O(\|T\|)$	Each transition visited exactly once
Total time	$O(n_{\text{iter}} \cdot \|T\|)$	$n_{\text{iter}} \approx 200$ for $\gamma = 0.9$
Space (index)	$O(\|T\|)$	Each transition stored once
Space (value fn)	$O(\|S\|)$	One double per state
Space (policy)	$O(\|S\|)$	One string per state

5.7 Test Results: gridworld_simple.mdp

5.7.1 Solver Output

Listing 5.3: Complete solver output for the 4-state MDP

```
Value Iteration converged after 242 iteration(s).
Final delta: 9.38e-10

State       R(s)      V*(s)       pi*(s)
---------------------------------------------
Trap       -50.0000  -500.0000   Stay
Goal       100.0000  1000.0000   Stay
Middle       1.0000   496.0000   MoveRight
Start        0.0000   435.5122   MoveRight
```

5.7.2 Manual Mathematical Verification

We verify each result against the Bellman equation by hand.

Absorbing states (exact closed-form):

$$V^*(\text{Goal}) = \frac{R(\text{Goal})}{1-\gamma} = \frac{100}{1-0.9} = \frac{100}{0.1} = 1000.0\checkmark$$

$$V^*(\text{Trap}) = \frac{R(\text{Trap})}{1-\gamma} = \frac{-50}{0.1} = -500.0\checkmark$$

Middle state:

$$Q(\text{Middle}, \text{MoveRight}) = 0.7 \times V^*(\text{Goal}) + 0.3 \times V^*(\text{Trap})$$

$$= 0.7 \times 1000 + 0.3 \times (-500) = 700 - 150 = 550$$

$$V^*(\text{Middle}) = R(\text{Middle}) + \gamma \times 550 = 1 + 0.9 \times 550 = 1 + 495 = 496.0\checkmark$$

Start state (approximate):

$$Q(\text{Start}, \text{MoveRight}) = 0.8 \times V^*(\text{Middle}) + 0.2 \times V^*(\text{Start})$$

This is self-referential — $V^*(\text{Start})$ appears on both sides. The iterative process converges to:

$$V^*(\text{Start}) \approx 435.5122\checkmark$$

Chapter 6

GridWorld Case Study: DSL vs. Hard-coded

6.1 The 4×3 GridWorld Problem

The 4×3 GridWorld is the canonical benchmark from Chapter 17 of *Artificial Intelligence: A Modern Approach* (Russell & Norvig). It is simple enough to reason about by hand, yet complex enough to illustrate the full power of MDP solvers.

6.1.1 Problem Specification

- **Grid:** 4 columns × 3 rows = 12 cells
- **Wall:** Cell (Row 2, Col 2) is blocked — not a state
- **Navigable states:** $12 - 1 = 11$
- **Goal:** (Row 3, Col 4) — reward +1.0, absorbing
- **Trap:** (Row 2, Col 4) — reward −1.0, absorbing
- **Living penalty:** $R(s) = -0.04$ for all other states
- **Actions:** Up, Down, Left, Right
- **Slip model:** 80% intended, 10% left, 10% right
- **Discount:** $\gamma = 0.9$
- **Total transitions:** 105

6.2 DSL Solution: gridworld_4x3.mdp

The MDP is defined in 259 lines of human-readable `.mdp` syntax. Key excerpts:

Listing 6.1: State and action declarations

```
# 11 navigable states (R2C2 is a WALL --- not declared)
STATE: R1C1
STATE: R1C2
STATE: R1C3
STATE: R1C4
STATE: R2C1
STATE: R2C3
STATE: R2C4
STATE: R3C1
STATE: R3C2
STATE: R3C3
STATE: R3C4

ACTION: Up
ACTION: Down
ACTION: Left
ACTION: Right
```

Listing 6.2: Rewards

```
REWARD: R1C1  -0.04
REWARD: R1C2  -0.04
REWARD: R1C3  -0.04
REWARD: R1C4  -0.04
REWARD: R2C1  -0.04
REWARD: R2C3  -0.04
REWARD: R2C4  -1.0      # Trap
REWARD: R3C1  -0.04
REWARD: R3C2  -0.04
REWARD: R3C3  -0.04
REWARD: R3C4   1.0      # Goal

DISCOUNT: 0.9
```

6.3 Hardcoded C++ Solution

The hardcoded version (`gridworld_hardcoded.cpp`) implements the same MDP using raw arrays and manual index arithmetic. Key differences:

Listing 6.3: Hardcoded transition logic (manual index arithmetic)

```
// State: flat index = row * COLS + col
int toIndex(int r, int c) { return r * COLS + c; }

// Get result state when trying action 'a' from cell (r,c)
int moveResult(int r, int c, int action) {
    int nr = r + DR[action];
    int nc = c + DC[action];
    if (!isValid(nr, nc)) return toIndex(r, c);  // Bounce back
    return toIndex(nr, nc);
}

// The stochastic Q-value: 80% intended + 10% left + 10% right
double q = 0.8 * V[s_intended]
         + 0.1 * V[s_slipL]
         + 0.1 * V[s_slipR];
```

6.4 Quantitative Comparison

Table 6.1: DSL pipeline vs. hardcoded C++ — complete comparison

Metric	**DSL Pipeline**	**Hardcoded C++**
Lines to define the MDP	259 (.mdp file, data only)	Embedded in code
Lines of solver code	0 (reused from compiler)	∼140
Total new code per new problem	259 lines (data only)	∼204 lines (data + logic)
Probability sum validation	Automatic (7 checks)	None
State name typo detection	Automatic	None
Solver reusable for new MDP?	**Yes**	No — full rewrite
Separation of data and logic	Complete	None
Convergence iterations	198	198
V^* results match?	Identical to 4 decimal places	

6.5 Solver Results: The Optimal Policy

Listing 6.4: DSL solver output for gridworld_4x3.mdp

```
Value Iteration converged after 198 iteration(s).
Final delta: 9.68e-10

State       R(s)      V*(s)       pi*(s)
--------------------------------------------
R3C1       -0.0400    6.3141      Right
R3C2       -0.0400    7.3490      Right
R3C3       -0.0400    8.4253      Right
R3C4        1.0000   10.0000     (absorb)
R2C1       -0.0400    5.4953      Up
R2C3       -0.0400    5.6332      Up
R2C4       -1.0000  -10.0000     (absorb)
R1C1       -0.0400    4.7080      Up
R1C2       -0.0400    4.0851      Left
R1C3       -0.0400    4.6195      Up
R1C4       -0.0400    2.6220      Left
```

6.5.1 Policy Visualisation

R3C1 $V = 6.31$ $\rightarrow$	R3C2 $V = 7.35$ $\rightarrow$	R3C3 $V = 8.43$ $\rightarrow$	**GOAL** R3C4 $V = 10.00$
R2C1 $V = 5.50$ $\uparrow$	WALL	R2C3 $V = 5.63$ $\uparrow$	**TRAP** R2C4 $V = -10.00$
R1C1 $V = 4.71$ $\uparrow$	R1C2 $V = 4.09$ $\leftarrow$	R1C3 $V = 4.62$ $\uparrow$	R1C4 $V = 2.62$ $\leftarrow$

Figure 6.1: Optimal policy visualisation on the 4×3 grid. Arrows show $\pi^*(s)$ for each state.

6.5.2 Policy Analysis

The route to the goal: Starting from R1C1, the optimal path is:

$$\text{R1C1} \xrightarrow{\uparrow} \text{R2C1} \xrightarrow{\uparrow} \text{R3C1} \xrightarrow{\rightarrow} \text{R3C2} \xrightarrow{\rightarrow} \text{R3C3} \xrightarrow{\rightarrow} \text{GOAL}$$

The policy routes the agent along the top row (Row 3), staying far from the trap in Row 2.

The interesting cases:

- **R1C2 goes Left** (not Up): Going Up from R1C2 would hit the wall and bounce back. Going Left reaches R1C1, which has a cleaner path up.
- **R1C4 goes Left** (not Up): Going Up from R1C4 would reach the Trap! Due to stochastic slipping, even if the agent tries to go Up correctly 80% of the time, the 10% left-slip from R2C4 could land in the trap. The optimal policy correctly avoids this.

Terminal state values:

$$V^*(\text{Goal}) = \frac{+1.0}{1 - 0.9} = 10.0 \qquad V^*(\text{Trap}) = \frac{-1.0}{1 - 0.9} = -10.0$$

Chapter 7

Edge Cases and Testing

7.1 Test Suite Overview

Table 7.1: Complete test matrix

Test File	What It Tests	Expected	Result
`gridworld_simple.mdp`	Valid 4-state MDP, full pipeline	PASS	✓
`gridworld_4x3.mdp`	11-state GridWorld with wall & slip	PASS	✓
`invalid_mdp.mdp`	5 planted errors	5 ERRORS caught	✓
`edge_cases.mdp`	Duplicate states, malformed lines	WARNINGS	✓
`edge_gamma1.mdp`	$\gamma = 1.0$ boundary (divergent)	Caps at 10K iter	✓
Missing file	Non-existent path	Fatal error, exit 1	✓
Empty file	No states parsed	Exit code 1	✓

7.2 The $\gamma = 1.0$ Edge Case

When $\gamma = 1.0$, the Bellman contraction mapping theorem's precondition ($\gamma < 1$) fails. The geometric series $\sum_{t=0}^{\infty} 1^t \cdot R$ diverges for any $R \neq 0$.

Why $\gamma = 1.0$ causes non-convergence

The contraction factor is $\gamma = 1.0$. Therefore:

$$\|V_{k+1} - V^*\|_\infty \leq 1.0 \cdot \|V_k - V^*\|_\infty$$

The distance to V^* does not shrink. Value Iteration runs the full 10,000 iterations without reaching $\delta < \theta$.

The compiler handles this gracefully: it runs all 10,000 iterations, prints a "did not con-

verge" message, and outputs whatever values were computed — without crashing or hanging indefinitely.

7.3 The Missing File Edge Case

If the user runs:

```
./mdp_compiler nonexistent.mdp
```

The `std::ifstream::is_open()` check immediately detects the failure, prints a clear error message with the filename, and exits with code 1. No undefined behaviour, no crash.

7.4 The Empty File Edge Case

A file with only blank lines and comments parses successfully (zero states, zero transitions) but fails Phase 2 validation (the MDP has no states — this is flagged as an error), and exits gracefully.

7.5 Duplicate State Declarations

```
STATE: Start
STATE: Start   # Duplicate!
```

Since `states` is an `unordered_set`, inserting a duplicate returns `ok = false`. The parser prints a warning but continues. Only one `Start` state exists in the AST.

7.6 Malformed TRANSITION Lines

```
TRANSITION: Start MoveRight   # Missing dest and prob!
```

After extracting `keyword`, `src`, and `act`, the `iss » dst` extraction fails because there is no fourth token. `iss.fail()` returns `true`, a warning is printed, and the line is skipped.

Chapter 8

Lessons Learned and Future Work

8.1 Engineering Lessons

8.1.1 Separation of Concerns Pays Off

The upfront investment in building a reusable compiler paid off immediately when defining the 4×3 GridWorld. The solver code was not touched at all — only a new `.mdp` file was written. Each new MDP problem requires only data, not code.

8.1.2 Validation Is Not Optional

The 7-check validator catches errors that would otherwise produce subtly wrong AI policies with no warning. In the hardcoded approach, a missing transition probability or a misspelled state name produces an incorrect solver result silently. In a real application (robot navigation, financial optimisation), this could have serious consequences.

8.1.3 Floating-Point Equality Is Never Safe

Direct equality checks (`sum == 1.0`) failed on completely valid MDPs due to IEEE 754 rounding. The $\varepsilon = 10^{-9}$ tolerance was essential. This lesson applies far beyond this project: any code that compares floating-point numbers for equality should use a tolerance.

8.1.4 The Transition Index Was Critical

Without the index, Value Iteration on the 4×3 GridWorld would scan 105 transitions per (s, a) query, per state, per iteration. With 11 states, 4 actions, and 198 iterations, that would be $11 \times 4 \times 105 \times 198 \approx 915{,}000$ operations. With the index, the inner loop cost

is proportional to only the 2–3 actual outcomes per (s, a) pair — roughly $100\times$ faster for this MDP.

8.1.5 RAII Prevents Entire Bug Classes

By using STL containers exclusively (no raw pointers, no manual memory management), memory leaks are structurally impossible. The effort of learning RAII and value semantics was repaid many times over in debugging time saved.

8.2 Future Work

8.2.1 State-Action Rewards: $R(s, a)$

The current DSL supports only state rewards $R(s)$. Many real-world MDPs have rewards that depend on both the state and the action taken: $R(s, a)$. This would require extending the grammar with a new keyword (`ACTION_REWARD:`), updating the AST struct, and modifying the Bellman update.

8.2.2 Policy Iteration

Policy Iteration is an alternative algorithm that converges in fewer iterations than Value Iteration for large MDPs, though each iteration is more expensive (it requires solving a linear system). Implementing both and comparing convergence rates would be a valuable extension.

8.2.3 Graphical Output

The solver currently prints results as text tables. Generating **Graphviz** `.dot` files would allow users to visualise the MDP graph and optimal policy as diagrams. Generating **TikZ** code would embed the policy diagram directly into LaTeX reports.

8.2.4 Larger GridWorlds and Performance Testing

Testing on 10×10 (99 navigable states), 20×20 (399 states), and 100×100 (9999 states) GridWorlds would reveal the practical performance limits of the current implementation and motivate further optimisations.

8.2.5 Interactive REPL Mode

A Read-Eval-Print Loop (REPL) would let users modify transitions or rewards interactively and immediately see the effect on the optimal policy — useful for teaching MDP

concepts.

8.2.6 Multi-Armed Bandits

The DSL could be extended to support stateless decision problems (bandit problems), demonstrating the language's generality beyond MDPs.

8.3 Conclusion

The MDP-DSL project demonstrates that a small, purpose-built language with automatic validation makes a measurable, practical difference in the correctness and usability of AI problem definitions.

The key results:

- A complete, working compiler pipeline in 763 lines of pure C++17
- 7 automatic validation checks that catch real errors (demonstrated on `invalid_mdp.mdp`)
- A correct Value Iteration solver (results verified mathematically by hand)
- Identical results to a carefully hand-coded C++ reference implementation on the 4×3 GridWorld
- Robust handling of all edge cases tested

The **core engineering insight**: by investing ~760 lines in a reusable compiler, every subsequent MDP can be defined purely as data — no algorithm code, no index arithmetic, no silent errors.

Final Takeaway

A domain-specific language is not just a convenience — it is a **correctness tool**. By restricting what can be said and automatically verifying what is said, a DSL catches an entire class of bugs before they become mysterious AI failures. This is the same principle behind type systems in general-purpose languages, schema validation in databases, and HTML validators on the web. MDP-DSL applies it to the domain of AI decision problems.

Appendix A

Complete File Inventory

Table A.1: All files in the MDP-DSL project

File	Purpose	Lines
Source Code (src/)		
`parser.cpp`	Phase 1 standalone parser	658
`mdp_compiler.cpp`	Unified pipeline (Phase 1+2+3)	763
`gridworld_hardcoded.cpp`	Phase 4 hardcoded comparison	204
Example MDP Files (examples/)		
`gridworld_simple.mdp`	4-state test MDP	44
`gridworld_4x3.mdp`	Classic 4×3 GridWorld	259
`invalid_mdp.mdp`	Validator error tests	49
`edge_cases.mdp`	Parser edge case tests	16
`edge_gamma1.mdp`	$\gamma = 1.0$ boundary test	12
Documentation (docs/)		
`phase1_report.pdf`	Parser Engine (15 pages)	—
`phase2_report.pdf`	Semantic Validator (12 pages)	—
`phase3_report.pdf`	Value Iteration Solver (10 pages)	—
`phase4_report.pdf`	GridWorld Case Study (8 pages)	—
`phase5_report.pdf`	Final Report (8 pages)	—
`README.md`	Project overview and quick start	—
Total C++ code: 1,625 lines; Total MDP definitions: 380 lines		

Appendix B

Quick-Reference: The .mdp Grammar

Listing B.1: A template .mdp file with all five keywords

```
# --- STATES ----------------------------------------
STATE: StateName1
STATE: StateName2
# Add as many states as needed

# --- ACTIONS ---------------------------------------
ACTION: ActionName1
ACTION: ActionName2

# --- TRANSITIONS -------------------------------------
# TRANSITION: <source> <action> <dest> <probability>
# For each (source, action) pair, all probs must sum to 1.0
TRANSITION: StateName1 ActionName1 StateName2 0.9
TRANSITION: StateName1 ActionName1 StateName1 0.1

# --- REWARDS ---------------------------------------
# REWARD: <state> <value>   (positive = reward, negative = penalty)
REWARD: StateName1  0.0
REWARD: StateName2  10.0

# --- DISCOUNT --------------------------------------
# Must be in [0.0, 1.0]. Standard values: 0.9, 0.95, 0.99
DISCOUNT: 0.9
```

Appendix C

Compilation and Execution Guide

C.1 Prerequisites

- A C++17-compatible compiler: `g++ 7+` (Linux/macOS), `clang++ 5+` (macOS), or MinGW-w64/WSL (Windows)
- No external libraries required — pure C++ standard library

C.2 Compilation

```
# Navigate to the source directory
cd src/

# Compile the unified pipeline (Phases 1 + 2 + 3)
g++ -std=c++17 -O2 -Wall -Wextra -o mdp_compiler mdp_compiler.cpp

# Compile the hardcoded comparison (Phase 4)
g++ -std=c++17 -O2 -Wall -Wextra -o hardcoded gridworld_hardcoded.cpp
```

Flag explanations:

- `-std=c++17` — required for structured bindings (`auto [it, ok] = ...`)
- `-O2` — optimisation level 2: faster binary, safe for correctness
- `-Wall -Wextra` — enables all compiler warnings; zero warnings confirms code quality

C.3 Running the Compiler

```
# Run on the 4-state test MDP
./mdp_compiler ../examples/gridworld_simple.mdp

# Run on the full 4x3 GridWorld
./mdp_compiler ../examples/gridworld_4x3.mdp

# Test the validator with a broken MDP
./mdp_compiler ../examples/invalid_mdp.mdp

# Test the gamma=1.0 edge case
./mdp_compiler ../examples/edge_gamma1.mdp

# Run the hardcoded comparison
./hardcoded
```

C.4 Expected Exit Codes

Scenario	Exit Code	Meaning
Successful solve	0	All phases passed, optimal policy printed
Validation errors	1	MDP is invalid — errors reported
File not found	1	Fatal file I/O error
Empty/no-state file	1	No MDP to solve

Appendix D

Glossary

AST Abstract Syntax Tree — the structured in-memory representation of parsed code.

Absorbing state A state from which no transitions lead elsewhere; the agent is permanently stuck.

Action A choice available to the agent; declared with `ACTION:`.

Bellman equation The recursive equation $V^*(s) = R(s) + \gamma \max_a \sum_{s'} T(s, a, s')V^*(s')$.

Contraction mapping A function that brings its inputs closer together; the Bellman operator is one when $\gamma < 1$, guaranteeing convergence.

Discount factor γ Controls how much future rewards are valued; declared with `DISCOUNT:`.

DSL Domain-Specific Language — a small language designed for one particular problem domain.

Expected value Probability-weighted average of possible outcomes.

GridWorld A rectangular grid where an agent navigates to a goal while avoiding traps; the standard MDP benchmark.

Lexer A program component that converts raw text into tokens.

Markov property The future depends only on the current state, not on history.

MDP Markov Decision Process — the tuple $\langle S, A, T, R, \gamma \rangle$.

Optimal policy π^* The policy that maximises expected discounted return from every state.

Optimal value function V^* The maximum expected discounted return achievable from each state.

Parser A program component that converts tokens into structured data (AST).

Policy π A mapping from states to actions — the agent's complete decision strategy.

Q-function $Q(s, a)$ Expected return from taking action a in state s then following π^*.

RAII Resource Acquisition Is Initialisation — a C++ idiom where resources are managed by object lifetimes.

Reward $R(s)$ Immediate numerical feedback for being in state s; declared with `REWARD:`.

Semantic validation Checking the meaning (not just structure) of parsed data for correctness.

State A complete description of the agent's current situation; declared with `STATE:`.

Stochastic Random; probabilistic rather than deterministic.

Transition $T(s, a, s')$ Probability of reaching state s' from s via action a; declared with `TRANSITION:`.

Value Iteration The iterative dynamic programming algorithm that converges to V^*.